Refinamento de Prompts em Desenvolvimento de Sistemas

1. Geração de Código

❖ Prompt inicial (vago):

Crie um código em C para calcular a soma de números fracionários

c Copy code

```c
#include <stdio.h>

int main() {
    // Declaração das variáveis para armazenar os números fracionários
    double num1, num2, soma;

    // Solicita ao usuário que insira o primeiro número
    printf("Digite o primeiro número fracionário: ");
    scanf("%lf", &num1);

    // Solicita ao usuário que insira o segundo número
    printf("Digite o segundo número fracionário: ");
    scanf("%lf", &num2);

    // Calcula a soma dos dois números
    soma = num1 + num2;

    // Exibe o resultado da soma
    printf("A soma de %.2lf e %.2lf é: %.2lf\n", num1, num2, soma);

    return 0;
}
```

Explicação:

- %lf é usado com scanf para ler valores do tipo double.
- %.2lf é usado com printf para exibir o número com duas casas decimais.

❖ Refinamento:

Crie um código em C que receba uma lista de números fracionários como entrada e retorne a soma. Inclua comentários explicativos em cada etapa e valide se a lista está vazia antes de calcular.

```c
#include <stdio.h>

int main() {
    double numero;   // Variável para armazenar cada número lido
    double soma = 0.0; // Inicializa a soma em zero
    int contador = 0;  // Contador para saber se a lista está vazia

    printf("Digite os números fracionários (um por linha).\n");
    printf("Para finalizar, digite Ctrl+D (Unix) ou Ctrl+Z (Windows) e pressione Enter:\n");

    // Loop para ler números até o fim da entrada (EOF)
    while (scanf("%lf", &numero) == 1) {
        soma += numero;   // Soma o número lido
        contador++;       // Incrementa o contador de números válidos
    }

    // Verifica se a lista está vazia
    if (contador == 0) {
        printf("A lista está vazia. Nenhum número foi digitado.\n");
    } else {
        printf("A soma dos números digitados é: %.2lf\n", soma);
    }

    return 0;
```

Comentários explicativos:

- **double numero;**
  Declara uma variável para armazenar cada número fracionário lido.
- **double soma = 0.0;**
  Inicializa a variável que vai acumular a soma dos números.
- **int contador = 0;**
  Contador para verificar se algum número foi digitado (evita dizer que a soma é zero quando a lista está vazia).
- **while (scanf("%lf", &numero) == 1)**
  Lê números enquanto houver entrada válida. O scanf retorna 1 quando consegue ler um número do tipo double.
- **soma += numero;**
  Soma o número lido à variável soma.
- **contador++;**
  Incrementa o contador para cada número válido lido.
- **if (contador == 0)**
  Verifica se nenhum número foi digitado. Se o contador for zero, informa que a lista está vazia.

Tarefa: Comparar os dois resultados e identificar como o refinamento melhora a utilidade do código.

1. Primeiro Código (Soma de dois números fracionários)

```c
#include <stdio.h>

int main() {
    double num1, num2, soma;

    printf("Digite o primeiro número fracionário: ");
    scanf("%lf", &num1);

    printf("Digite o segundo número fracionário: ");
    scanf("%lf", &num2);

    soma = num1 + num2;

    printf("A soma de %.2lf e %.2lf é: %.2lf\n", num1, num2, soma);

    return 0;
}
```

Limitações:

- Só aceita exatamente dois números.
- Não permite entrada de listas ou quantidades variáveis de números.
- Não verifica se a entrada está vazia ou se o usuário não digitou nada.
- Pouco flexível para uso em cenários reais onde se deseja somar vários números.

---

2. Segundo Código (Soma de uma lista de números fracionários com validação)

```c
#include <stdio.h>

int main() {
    double numero;
    double soma = 0.0;
    int contador = 0;

    printf("Digite os números fracionários (um por linha).\n");
    printf("Para finalizar, digite qualquer caractere não numérico ou pressione Ctrl+D (Unix) / Ctrl+Z (Win

    while (scanf("%lf", &numero) == 1) {
        soma += numero;
        contador++;
    }

    if (contador == 0) {
        printf("A lista está vazia. Nenhum número foi informado.\n");
    } else {
        printf("A soma dos números informados é: %.2lf\n", soma);
    }

    return 0;
}
```

Melhorias e utilidade:

- **Aceita uma quantidade arbitrária de números fracionários.**
- **Permite ao usuário digitar quantos números quiser, até encerrar a entrada.**
- **Valida se a lista está vazia e informa ao usuário caso nenhum número tenha sido digitado.**
- **Mais flexível e útil para aplicações reais, como cálculo de médias, somas de notas, etc.**
- **Evita erros de execução ou resultados inesperados por falta de entrada.**

Comparação e análise do refinamento

Critério	Primeiro Código	Segundo Código (refinado)
Quantidade de entradas	Fixa (2)	Variável (quantos quiser)
Validação de entrada	Não faz	Sim (verifica se está vazia)
Flexibilidade	Baixa	Alta
Utilidade prática	Limitada	Alta (aplicável em muitos casos)
Experiência do usuário	Ruim (só dois números)	Melhor (pode digitar vários)

Conclusão

O **refinamento** do código:

- **Aumenta a flexibilidade**, permitindo que o usuário insira quantos números desejar.
- **Melhora a robustez**, pois verifica se a entrada está vazia e informa ao usuário.
- **Torna o código mais útil** para situações reais, onde normalmente não se sabe de antemão quantos números serão somados.
- **Reduz a chance de erros** ou resultados inesperados.

O segundo código é **muito mais prático, robusto e amigável ao usuário** do que o primeiro, mostrando como pequenas melhorias de lógica e validação podem transformar a utilidade de um programa.